Diffusion Model Mathematics Notes: VP-SDE, DDPM, and DDIM Sampling

Rich Wu

June 11, 2026

Abstract

These notes summarize the mathematical basis of the TensorFlow DDIM implementation in this repository. The presentation starts from the variance-preserving stochastic differential equation (VP-SDE), derives the closed-form forward transition, derives the Gaussian posterior used by DDPM, and then rewrites the reverse step in the DDIM parameterization used for fast sampling. The notation is chosen to match the implementation: α_t denotes the cumulative signal power, $1 - \alpha_t$ denotes the noise power, and the model may predict noise, clean image, or velocity.

1 Introduction

Denoising diffusion models learn to invert a gradual noising process. A clean sample $\mathbf{x}_0$ from the data distribution is transformed into a noisy sample $\mathbf{x}_t$ by mixing it with Gaussian noise. The learning problem is to train a neural network to estimate the clean signal or the injected noise from the noisy observation. Once trained, the model generates new samples by starting from Gaussian noise and repeatedly applying a learned reverse transition.

This document focuses on the variance-preserving family used by DDPM and DDIM. In this convention the forward marginal has the form

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\sqrt{\alpha_t}\,\mathbf{x}_0, (1-\alpha_t)\mathbf{I}), \tag{1}$$

where $\alpha_0 = 1$ and α_t decreases toward zero as t approaches the terminal diffusion time. Many papers denote this cumulative signal power by $\bar{\alpha}_t$; here we write α_t to match the code.

2 Notation and Time Indexing

The random variable at diffusion time t is denoted by $\mathbf{X}(t)$, and a realized sample is denoted by $\mathbf{x}_t$. For $0 \le s < t \le T$, $\mathbf{x}_s$ is less noisy than $\mathbf{x}_t$. The implementation discretizes time into integer indices $0, 1, \ldots, N$, but the derivation is easiest in continuous time. The two are connected by evaluating the continuous schedule at a finite grid.

3 Forward Process from the VP-SDE

The variance-preserving SDE is

$$d\mathbf{X}(t) = -\frac{\beta(t)}{2}\mathbf{X}(t)\,dt + \sqrt{\beta(t)}\,d\mathbf{W}(t), \tag{2}$$

where $\beta(t) \ge 0$ is the noise-rate schedule and $\mathbf{W}(t)$ is a standard Wiener process. Informally, $d\mathbf{W}(t)$ is a Gaussian increment with covariance $dt\,\mathbf{I}$, so the drift term shrinks the signal while the diffusion term injects Gaussian noise.

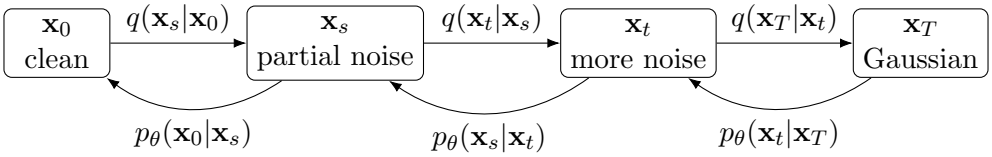

Figure 1: The forward process is analytically known and gradually destroys signal. The reverse process is learned and generates samples by moving from Gaussian noise back to data space.

Table 1: Main symbols used in this note and in the implementation.

Symbol	Meaning
α_t	cumulative signal power at time t
$1-\alpha_t$	cumulative noise power at time t
$\sqrt{\alpha_t}$	signal coefficient used in forward sampling
$\sqrt{1-\alpha_t}$	noise coefficient used in forward sampling
$\boldsymbol{\epsilon}$	standard Gaussian noise, $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
$\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$	neural network estimate of the forward noise
$\mathbf{x}_0^\theta(\mathbf{x}_t, t)$	neural network implied estimate of the clean sample
$\Sigma_{s,t}^2$	DDPM posterior variance for stepping from t to s
η	DDIM stochasticity parameter; $\eta = 0$ is deterministic

For the scalar case, the transition density satisfies the Fokker–Planck equation

$$\frac{\partial}{\partial t} q(x_t|x_s) = \frac{\beta(t)}{2} \left[\frac{\partial}{\partial x_t} \left(x_t q(x_t|x_s) \right) + \frac{\partial^2 q(x_t|x_s)}{\partial x_t^2} \right]. \tag{3}$$

For vector-valued images, this equation is interpreted component-wise or, equivalently, with divergence and Laplacian operators:

$$\frac{\partial}{\partial t} q(\mathbf{x}_t|\mathbf{x}_s) = \frac{\beta(t)}{2} \left[\nabla_{\mathbf{x}_t} \cdot (\mathbf{x}_t q(\mathbf{x}_t|\mathbf{x}_s)) + \Delta_{\mathbf{x}_t} q(\mathbf{x}_t|\mathbf{x}_s) \right]. \tag{4}$$

3.1 Closed-Form Forward Transition

Define the integrated noise rate and cumulative signal power as

$$B(t) = \int_0^t \beta(\tau)\, d\tau, \tag{5}$$

$$\alpha_t = \exp(-B(t)). \tag{6}$$

Solving the linear SDE gives the Gaussian transition

$$q(\mathbf{x}_t|\mathbf{x}_s) = \mathcal{N}\left(\sqrt{\frac{\alpha_t}{\alpha_s}}\, \mathbf{x}_s, \left(1 - \frac{\alpha_t}{\alpha_s} \right) \mathbf{I} \right), \qquad 0 \le s < t. \tag{7}$$

Setting $s = 0$ and $\alpha_0 = 1$ gives the forward marginal

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\sqrt{\alpha_t}\, \mathbf{x}_0, (1-\alpha_t)\mathbf{I}). \tag{8}$$

Therefore a noisy sample can be generated directly at any time:

$$\mathbf{x}_t = \sqrt{\alpha_t}\, \mathbf{x}_0 + \sqrt{1-\alpha_t}\, \boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \tag{9}$$

This direct formula is what makes training efficient: the model does not need to simulate every intermediate noising step.

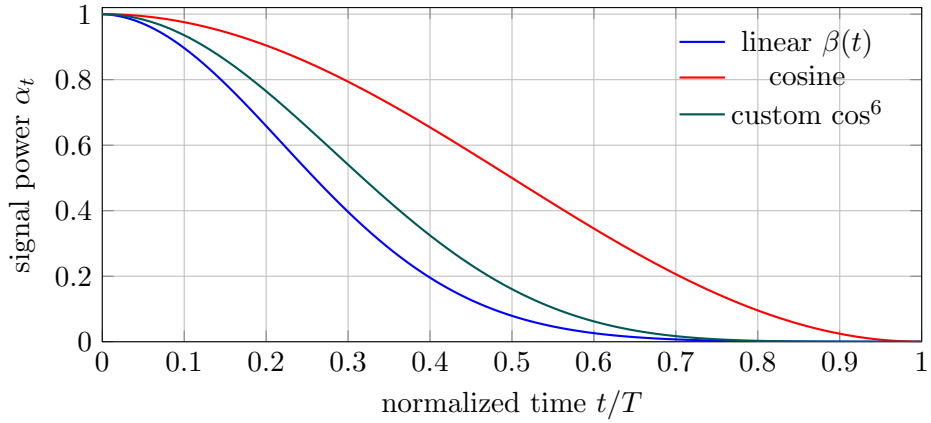

Figure 2: Examples of cumulative signal-power schedules. Faster decay produces lower SNR earlier in the forward process and changes which noise levels dominate training.

3.2 Noise Schedules

The schedule controls the signal-to-noise ratio

$$\mathrm{SNR}(t) = \frac{\alpha_t}{1-\alpha_t}. \tag{10}$$

A linear continuous schedule for $\beta(t)$ over normalized time $t \in [0,1]$ is

$$\beta(t) = \beta_0 + t(\beta_1 - \beta_0), \tag{11}$$

$$\alpha_t = \exp\left[-\beta_0 t - \frac{1}{2}(\beta_1 - \beta_0)t^2\right]. \tag{12}$$

A simple cosine schedule can be written as

$$\alpha_t = \cos^2\left(\frac{\pi}{2}\frac{t}{T}\right), \qquad 0 \le t \le T. \tag{13}$$

The implementation also includes the improved DDPM cosine schedule, which discretizes an offset cosine curve into per-step betas and then takes their cumulative product.

4 Training Parameterizations

Equation (9) gives a useful identity between the clean sample, noise, and noisy sample. The network can be trained to predict different but equivalent targets.

4.1 Noise Prediction

The classic DDPM target is the injected noise:

$$\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \approx \boldsymbol{\epsilon}. \tag{14}$$

Given a noise prediction, the implied clean-sample estimate is

$$\mathbf{x}_0^\theta(\mathbf{x}_t, t) = \frac{\mathbf{x}_t - \sqrt{1-\alpha_t}\,\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)}{\sqrt{\alpha_t}}. \tag{15}$$

4.2 Clean-Image Prediction

The model may also predict $\mathbf{x}_0$ directly:

$$\mathbf{x}_0^\theta(\mathbf{x}_t, t) \approx \mathbf{x}_0. \tag{16}$$

The corresponding noise estimate is

$$\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) = \frac{\mathbf{x}_t - \sqrt{\alpha_t}\,\mathbf{x}_0^\theta(\mathbf{x}_t, t)}{\sqrt{1-\alpha_t}}. \tag{17}$$

4.3 Velocity Prediction

The implementation supports velocity prediction, a parameterization used by Salimans and Ho for stable fast sampling and progressive distillation [4], defined by

$$\mathbf{v}_t = \sqrt{\alpha_t}\,\boldsymbol{\epsilon} - \sqrt{1-\alpha_t}\,\mathbf{x}_0. \tag{18}$$

This parameterization is numerically useful because it rotates the pair $(\mathbf{x}_0, \boldsymbol{\epsilon})$ into coordinates that remain well-scaled across noise levels. If the network predicts $\mathbf{v}_\theta$, the corresponding clean and noise estimates are

$$\mathbf{x}_0^\theta = \sqrt{\alpha_t}\,\mathbf{x}_t - \sqrt{1-\alpha_t}\,\mathbf{v}_\theta, \tag{19}$$

$$\boldsymbol{\epsilon}_\theta = \sqrt{\alpha_t}\,\mathbf{v}_\theta + \sqrt{1-\alpha_t}\,\mathbf{x}_t. \tag{20}$$

These are the conversions used by `DiffusionUtility.get_pred_components`.

5 Reverse Posterior for DDPM

The true reverse conditional $q(\mathbf{x}_s|\mathbf{x}_t)$ depends on the unknown data distribution and is not available in closed form. However, the posterior conditioned on the original clean sample is available:

$$q(\mathbf{x}_s|\mathbf{x}_t, \mathbf{x}_0) = \frac{q(\mathbf{x}_t|\mathbf{x}_s)q(\mathbf{x}_s|\mathbf{x}_0)}{q(\mathbf{x}_t|\mathbf{x}_0)}. \tag{21}$$

Because all three terms are Gaussian, this posterior is also Gaussian:

$$q(\mathbf{x}_s|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}\big(\boldsymbol{\mu}_{s,t}(\mathbf{x}_t, \mathbf{x}_0), \Sigma_{s,t}^2\mathbf{I}\big), \qquad 0 \le s < t. \tag{22}$$

The mean and variance are

$$\boldsymbol{\mu}_{s,t}(\mathbf{x}_t, \mathbf{x}_0) = \frac{1-\alpha_s}{1-\alpha_t}\sqrt{\frac{\alpha_t}{\alpha_s}}\,\mathbf{x}_t + \frac{1-\alpha_t/\alpha_s}{1-\alpha_t}\sqrt{\alpha_s}\,\mathbf{x}_0, \tag{23}$$

$$\Sigma_{s,t}^2 = \frac{1-\alpha_s}{1-\alpha_t}\left(1 - \frac{\alpha_t}{\alpha_s}\right). \tag{24}$$

For adjacent discrete DDPM steps, $s = t-1$. For accelerated sampling, s can skip multiple training indices, as long as $0 \le s < t$.

5.1 DDPM Sampling Step

DDPM replaces the unknown $\mathbf{x}_0$ in the posterior mean with the model estimate $\mathbf{x}_0^\theta$:

$$\mathbf{x}_s = \frac{1-\alpha_s}{1-\alpha_t}\sqrt{\frac{\alpha_t}{\alpha_s}}\,\mathbf{x}_t + \frac{1-\alpha_t/\alpha_s}{1-\alpha_t}\sqrt{\alpha_s}\,\mathbf{x}_0^\theta(\mathbf{x}_t, t) + \Sigma_{s,t}\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \tag{25}$$

This stochastic transition samples from the learned approximation to the DDPM reverse chain.

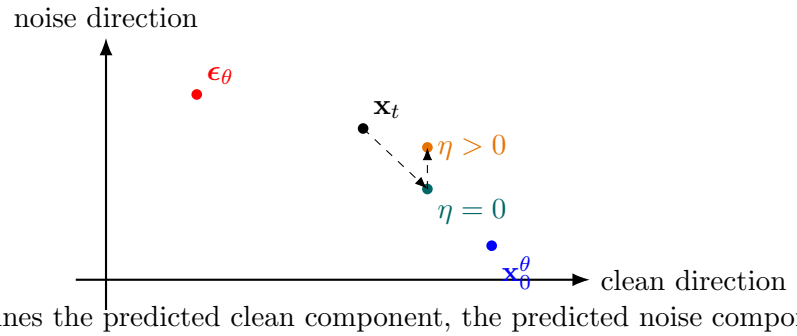

Figure 3: Schematic view of a DDIM step. The deterministic part moves along the subspace spanned by the predicted clean sample and predicted noise. The stochastic term adds fresh Gaussian noise with standard deviation $\eta\Sigma_{s,t}$.

6 DDIM Reverse Sampling

DDIM rewrites the reverse transition so that the predicted clean image and predicted noise appear explicitly:

$$\mathbf{x}_s = \sqrt{\alpha_s}\,\mathbf{x}_0^\theta(\mathbf{x}_t, t) + \sqrt{1 - \alpha_s - \sigma_{s,t}^2}\,\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) + \sigma_{s,t}\boldsymbol{\epsilon}, \tag{26}$$

where

$$\sigma_{s,t} = \eta\,\Sigma_{s,t}. \tag{27}$$

Substituting Equation (27) gives the implementation form:

$$\mathbf{x}_s = \sqrt{\alpha_s}\,\mathbf{x}_0^\theta(\mathbf{x}_t, t) + \sqrt{1 - \alpha_s - \eta^2\Sigma_{s,t}^2}\,\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) + \eta\Sigma_{s,t}\boldsymbol{\epsilon}. \tag{28}$$

Two important cases are:

$$\eta = 1, \qquad \text{DDPM-equivalent posterior variance,} \tag{29}$$

$$\eta = 0, \qquad \text{deterministic DDIM sampling.} \tag{30}$$

For intermediate values $0 < \eta < 1$, DDIM interpolates between deterministic sampling and stochastic DDPM-like sampling.

7 Algorithmic Summary

Training uses random diffusion times and the closed-form noising equation.

1. Sample clean data $\mathbf{x}_0$ from the dataset.
2. Sample a timestep t and Gaussian noise $\boldsymbol{\epsilon}$.
3. Form $\mathbf{x}_t = \sqrt{\alpha_t}\mathbf{x}_0 + \sqrt{1 - \alpha_t}\boldsymbol{\epsilon}$.
4. Train the network to predict the chosen target: $\boldsymbol{\epsilon}$, $\mathbf{x}_0$, or $\mathbf{v}_t$.

Sampling starts from Gaussian noise and applies Equation (28).

1. Sample $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.
2. Choose a descending sequence $T = t_K > t_{K-1} > \cdots > t_0 = 0$.

3. At each step, evaluate the model at $\mathbf{x}_{t_k}$ and convert its output into $\mathbf{x}_0^\theta$ and $\boldsymbol{\epsilon}_\theta$.

4. Compute $\mathbf{x}_{t_{k-1}}$ using the DDIM formula with $s = t_{k-1}$ and $t = t_k$.

The number of sampling steps K may be much smaller than the number of training timesteps, which is the main practical advantage of DDIM-style generation.

8 Implementation Correspondence

The implementation in `src/fddim/diffusion_utils.py` follows these equations directly.

Table 2: Mapping between equations and implementation variables.

Mathematical quantity	Code variable	Location
α_t	`self.alphas[t]`	`_compute_alphas`
$\sqrt{\alpha_t}$	`mu_coefs`	`_compute_forward_coefficients`
$\sqrt{1-\alpha_t}$	`sigma_coefs`	`_compute_forward_coefficients`
$\Sigma_{s,t}^2$	`reverse_var_coefs`	`_compute_reverse_coefficients`
$\sqrt{\alpha_s}$	`reverse_mu_ddim_x0`	`_compute_reverse_coefficients`
$\sqrt{1-\alpha_s-\eta^2\Sigma_{s,t}^2}$	`reverse_mu_ddim_noise`	`_compute_reverse_coefficients`
$\eta\Sigma_{s,t}$	`_sigma`	`p_sample_ddim`

One practical detail is clipping. If $\mathbf{x}_0^\theta$ is clipped to the valid image range, the predicted noise should be recomputed from Equation (17). This keeps the pair $(\mathbf{x}_0^\theta, \boldsymbol{\epsilon}_\theta)$ consistent with the observed $\mathbf{x}_t$.

9 Numerical Notes

Several edge cases matter in code:

- At $t = 0$, $\sqrt{1-\alpha_t} = 0$, so formulas that divide by this term need a small numerical guard or should avoid noise reconstruction at exactly zero.
- At very late times, α_t can be close to zero, so reconstructing $\mathbf{x}_0^\theta$ from a noise prediction can amplify model error.
- The square-root argument in Equation (28) must be non-negative. The standard DDIM choice $\sigma_{s,t} = \eta\Sigma_{s,t}$ with $0 \le \eta \le 1$ satisfies this for valid schedules.
- If the sampler skips many training indices, the neural network is asked to make larger reverse moves. This is faster, but sample quality may depend more strongly on the schedule and model parameterization.

10 Conclusion

The key identity behind both DDPM and DDIM is the closed-form forward marginal $\mathbf{x}_t = \sqrt{\alpha_t}\mathbf{x}_0 + \sqrt{1-\alpha_t}\boldsymbol{\epsilon}$. DDPM uses the Gaussian posterior $q(\mathbf{x}_s|\mathbf{x}_t, \mathbf{x}_0)$ and substitutes a learned clean-sample estimate. DDIM rearranges the same quantities into a step involving $\mathbf{x}_0^\theta$, $\boldsymbol{\epsilon}_\theta$, and a tunable stochasticity parameter η. With $\eta = 0$, sampling becomes deterministic and can use a small number of reverse steps; with $\eta = 1$, the variance matches the DDPM posterior variance.

References

[1] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising Diffusion Probabilistic Models. *Advances in Neural Information Processing Systems*, 2020.

[2] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising Diffusion Implicit Models. *International Conference on Learning Representations*, 2021.

[3] Alex Nichol and Prafulla Dhariwal. Improved Denoising Diffusion Probabilistic Models. *International Conference on Machine Learning*, 2021.

[4] Tim Salimans and Jonathan Ho. Progressive Distillation for Fast Sampling of Diffusion Models. *International Conference on Learning Representations*, 2022.

[5] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-Based Generative Modeling through Stochastic Differential Equations. *International Conference on Learning Representations*, 2021.